Automated Lower Bounds for Bilinear Complexity over Finite Fields

Chengu Wang
wangchengu@gmail.com

Abstract

We present a general, automated framework for proving lower bounds on the bilinear complexity (tensor rank) of multiplication problems over a finite field $\mathbb{F}_q$. The framework is parameterized only by the multiplication tensor and by a group of rank-preserving symmetries acting on one argument: it classifies the orbits of constraint subspaces under that group, runs a dynamic program over the orbits combining four lower-bound techniques, and emits a proof certificate that a verifier rechecks, typically faster than the search.

Instantiating the framework for matrix multiplication, we improve the lower bounds for four small formats over $\mathbb{F}_2$, most notably showing that the bilinear complexity of multiplying two 3×3 matrices over $\mathbb{F}_2$ is at least 20, raising the bound of 19 that had stood since Bläser (2003). Instantiating it for polynomial multiplication — full products, cyclic convolution, and the truncated (modulo x^N) and negacyclic (modulo $x^N + 1$) products — we obtain eighteen new lower bounds over $\mathbb{F}_2$ and $\mathbb{F}_3$. Every bound is backed by a machine-checkable certificate.

1 Introduction

A *bilinear* algorithm for a bilinear map $\beta\colon U \times V \to W$ over a field $\mathbb{F}$ computes a fixed set of products of a linear form in the U-variables with a linear form in the V-variables, and then reads off each coordinate of the output as a linear combination of those products. The minimum number of such products is the *bilinear complexity* of β, and it equals the rank of the order-3 tensor naturally associated with β. Two families of bilinear maps have driven the subject: matrix multiplication and polynomial multiplication. For matrix multiplication the central quantity is $\mathbf{R}(\langle l, m, n\rangle)$, the rank of the tensor for multiplying an $l \times m$ by an $m \times n$ matrix; for polynomial multiplication it is the rank of the tensor for multiplying two polynomials.

Over small finite fields these ranks are notoriously hard to pin down, and even for tiny formats the best known lower and upper bounds often disagree. This paper develops one automated method that attacks both families, and uses it to push several of these small cases.

1.1 Lower bounds for matrix multiplication

Strassen [30] showed $\mathbf{R}(\langle 2, 2, 2\rangle) \leq 7$ over $\mathbb{Z}$, and Winograd [33] proved the matching lower bound over $\mathbb{Q}$. Hopcroft and Kerr [21] then established several lower bounds over $\mathbb{F}_2$, including $\mathbf{R}_{\mathbb{F}_2}(\langle 2, 3, 3\rangle) \geq 15$, and reported that a detailed analysis "seems to indicate" $\mathbf{R}_{\mathbb{F}_2}(\langle 2, 3, 4\rangle) \geq 19$, without publishing a proof, while Bshouty [10] proved $\mathbf{R}_{\mathbb{F}_2}(\langle n, n, n\rangle) \geq \frac{5}{2}n^2 - o(n^2)$. Bläser established the influential bounds $\mathbf{R}(\langle l, m, n\rangle) \geq lm+mn+l-m+n-3$ over algebraically closed fields [7],

$\mathbf{R}(\langle n,n,n\rangle) \geq \frac{5}{2}n^2 - 3n$ over arbitrary fields [6], and finally $\mathbf{R}(\langle n,m,n\rangle) \geq 2mn + 2n - m - 2$ for $m \geq n \geq 3$ [8], which implies $\mathbf{R}(\langle 3,3,3\rangle) \geq 19$. On the upper side Laderman [24] multiplied 3×3 matrices with 23 products over $\mathbb{Z}$, so the rank of $\langle 3,3,3\rangle$ has remained trapped in $[19, 23]$, the prototypical open small format.

1.2 Lower bounds for polynomial multiplication

The bilinear complexity of polynomial multiplication has an equally long history and, unlike matrix multiplication, is governed by the arithmetic of $\mathbb{F}_q$. The *full* product is the base case of fast integer and polynomial multiplication. For two degree-$(N-1)$ polynomials, Fiduccia and Zalcstein [16] gave the lower bound $2N-1$, met by Toom–Cook evaluation–interpolation whenever $2N-1 \leq q+1$ [31]. When the field is too small for plain interpolation, the best upper bounds come from Karatsuba-like formulae, notably Montgomery's [26], and from CRT/evaluation–interpolation over higher-degree places [34]. The exact value then climbs: Kaminski and Bshouty [22] settled the next range, and for larger N the strongest lower bounds embed a bilinear algorithm into a linear code and invoke the Griesmer bound [18] or Grassl's tables [17], via the code constructions of Lempel–Seroussi–Winograd [25] and Chudnovsky–Chudnovsky [13]; small cases were settled by the exhaustive search of Barbulescu, Detrey, Estibals, and Zimmermann [4].

A second strand is multiplication in a polynomial quotient ring $\mathbb{F}_q[x]/(x^N - \gamma)$, which specializes to three classical families: *cyclic* convolution ($\gamma = 1$, modulo $x^N - 1$), the group algebra of a cyclic group, underlying the discrete Fourier transform and cyclic codes; the *truncated* (or short) product ($\gamma = 0$, modulo x^N), the local algebra $\mathbb{F}_q[x]/x^N$ that is the recurring base case of Karatsuba- and Toom-style schemes; and the *negacyclic* product ($\gamma = -1$, modulo $x^N + 1$), a twisted group algebra underlying skew-cyclic codes and lattice-based cryptography. The lower bounds for all three are governed by the factorization of $x^N - \gamma$: Winograd [34] proved that multiplication modulo a polynomial with k distinct irreducible factors needs at least $2N - k$ products (a bound Alder and Strassen [1] later extended to all associative algebras), and Bläser's characterization of algebras of minimal bilinear complexity [9], combined with the minimal-rank theory of Averbuch, Galil, and Winograd [2, 3] (whose Part II treats exactly $\mathbb{F}_q[x]/x^N$), strengthens these on the local-algebra factors. Upper bounds come from the convolution constructions of Wagh and Morgera [32] and Morgera [27], refined by Cenk and Özbudak [11, 12].

1.3 Our contributions

We give a single framework, described abstractly in Section 3 and instantiated per problem in Section 4, that proves bilinear-complexity lower bounds for any multiplication tensor equipped with a group of first-argument symmetries. It classifies the orbits of constraint subspaces (Section 5), runs a dimension-sweeping dynamic program with four lower-bound techniques (Section 6), and produces certificates that an independent verifier rechecks deterministically (Section 7).

Our main results concern matrix multiplication, where the framework improves the best known lower bound for four small formats over $\mathbb{F}_2$.

Theorem 1 (Matrix multiplication)**.**

$$\mathbf{R}_{\mathbb{F}_2}(\langle 2,3,4\rangle) \geq 19, \quad \mathbf{R}_{\mathbb{F}_2}(\langle 3,3,3\rangle) \geq 20, \quad \mathbf{R}_{\mathbb{F}_2}(\langle 3,3,4\rangle) \geq 25, \quad \mathbf{R}_{\mathbb{F}_2}(\langle 3,4,4\rangle) \geq 29.$$

The headline is $\mathbf{R}_{\mathbb{F}_2}(\langle 3,3,3\rangle) \geq 20$, which raises the value 19 that had stood since Bläser (2003) [8];

the search finds this proof in about 40 minutes on a laptop and the certificate verifies in seconds. The same certificate has since been re-verified independently by Beuchert [5], using a checker written from scratch in a different programming language. The bound $\mathbf{R}_{\mathbb{F}_2}(\langle 2,3,4\rangle) \geq 19$ confirms a claim that Hopcroft and Kerr [21] reported without proof.

For polynomial multiplication the same framework yields eighteen new lower bounds, over $\mathbb{F}_2$ and $\mathbb{F}_3$ — for the full product and for the three quotient families $\mathbb{F}_q[x]/(x^N - \gamma)$.

Theorem 2 (Polynomial multiplication)**.** *Write $\mathbf{R}_{\mathbb{F}_q}(\mathrm{P}_N)$, $\mathbf{R}_{\mathbb{F}_q}(\mathrm{C}_N)$, $\mathbf{R}_{\mathbb{F}_q}(\mathrm{T}_N)$, and $\mathbf{R}_{\mathbb{F}_q}(\mathrm{C}_N^-)$ for the bilinear complexity over $\mathbb{F}_q$ of, respectively, the full product of two degree-$(N-1)$ polynomials, cyclic convolution (modulo $x^N - 1$), the truncated product (modulo x^N), and the negacyclic product (modulo $x^N + 1$), each of length N. Then*

full: $\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_6) \geq 16$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_7) \geq 19$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_8) \geq 21$, $\mathbf{R}_{\mathbb{F}_3}(\mathrm{P}_6) \geq 14$;

cyclic: $\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_7) \geq 13$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_8) \geq 19$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_{10}) \geq 22$, $\mathbf{R}_{\mathbb{F}_3}(\mathrm{C}_9) \geq 19$;

truncated: $\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_6) \geq 13$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_7) \geq 16$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_8) \geq 19$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_9) \geq 21$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_{10}) \geq 24$, $\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_{11}) \geq 27$, $\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_8) \geq 17$, $\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_9) \geq 19$, $\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_{10}) \geq 21$;

negacyclic: $\mathbf{R}_{\mathbb{F}_3}(\mathrm{C}_9^-) \geq 19$.

family	problem	prev LB	our LB	prev UB
Matrix	$\mathbf{R}_{\mathbb{F}_2}(\langle 2,3,4\rangle)$	18 [7], 19? [21]	19	20 [21]
	$\mathbf{R}_{\mathbb{F}_2}(\langle 3,3,3\rangle)$	19 [8]	20	23 [24]
	$\mathbf{R}_{\mathbb{F}_2}(\langle 3,3,4\rangle)$	24 [8]	25	29 [29]
	$\mathbf{R}_{\mathbb{F}_2}(\langle 3,4,4\rangle)$	28 [7]	29	38 [29]
Full	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_6)$	15 [25, 13, 17]	16	17 [26]
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_7)$	18 [25, 13, 17]	19	22 [26]
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_8)$	20 [25, 13, 17]	21	26 [34, 15]
	$\mathbf{R}_{\mathbb{F}_3}(\mathrm{P}_6)$	12 [25, 13, 18]	14	15 [34]
Cyclic	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_7)$	12 [14, 9]	13	13 [32]
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_8)$	16 [3, 9]	19	22 [27, 12]
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_{10})$	19 [2, 9]	22	29 [27]
	$\mathbf{R}_{\mathbb{F}_3}(\mathrm{C}_9)$	18 [3, 9]	19	27 [27, 12]
Truncated	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_6)$	12 [3, 9]	13	14 [12]
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_7)$	14 [3, 9]	16	18 [12]
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_8)$	16 [3, 9]	19	22 [12]
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_9)$	18 [3, 9]	21	27 [12]*
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_{10})$	20 [3, 9]	24	31 [12]
	$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_{11})$	22 [3, 9]	27	36 [12]
	$\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_8)$	16 [3, 9]	17	22 [12]
	$\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_9)$	18 [3, 9]	19	27 [34]*
	$\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_{10})$	20 [3, 9]	21	31 [34]
Negacyclic	$\mathbf{R}_{\mathbb{F}_3}(\mathrm{C}_9^-)$	18 [3, 9]	19	27 [12]

* We obtain a new bilinear complexity upper bound of 26 for multiplication over $\mathbb{Z}[x]/x^9$ by flip-graph search [23] on $\mathbb{F}_2$ and lifting to $\mathbb{Z}$. See Table 6 for the decomposition.

Table 1: The 22 new lower bounds (4 matrix, 18 polynomial) proved in this paper.

Table 1 collects all twenty-two new lower bounds against the previous best lower bound and the

best known upper bound. Tables 3–7 in Appendix A set these in the full context of each family, including the many small cases where our framework reproduces the exact rank already known.

The rest of the paper is organized as follows. Section 2 sketches the framework on a toy example; Sections 3–7 develop it in full — setup, per-problem symmetries, orbit enumeration, the four lower-bound techniques, and certificates — and Section 8 concludes.

2 A toy example: $\mathbf{R}_{\mathbb{F}_2}(\langle 2,2,2\rangle) \geq 7$

Before the formal development, we sketch how the framework proves an easy case: that 2×2 matrix multiplication over $\mathbb{F}_2$ needs at least 7 multiplications. Everything below is made precise in Sections 3–7.

Write the tensor as $\langle 2,2,2\rangle = \sum_{i,j,k} a_{ij} \otimes b_{jk} \otimes c_{ki}$, where the a_{ij} are the four entries of the first matrix factor A. The method works with *constrained* versions of the tensor, obtained by forcing some linear combinations of the a_{ij} to vanish — that is, by restricting A to a subspace, and keeping only the terms with a surviving a_{ij}. The matrix symmetries $A \mapsto PAQ^{-1}$ and $A \mapsto A^\top$ permute these subspaces, and for 2×2 matrices over $\mathbb{F}_2$ there are only ten orbits up to symmetry, from the fully constrained $A = 0$ (orbit 0) up to the unconstrained tensor (orbit 9), which is $\langle 2,2,2\rangle$ itself.

The algorithm assigns each orbit a rank lower bound, working from the most constrained to the least so that each step may rely on the bounds already settled for smaller subspaces. We walk through all ten below, grouped by technique.

Orbits 0, 1, 2, 3, 5 (flatten). The base technique reads a bound off a *flattening*: regroup the tensor's three factors as two, view the result as a matrix, and take its rank. Orbit 0, $A = \left(\begin{smallmatrix} 0 & 0 \\ 0 & 0 \end{smallmatrix}\right)$, has constrained tensor 0, so $\mathbf{R} = 0$; orbit 1, $A = \left(\begin{smallmatrix} 0 & 0 \\ 0 & a_{11} \end{smallmatrix}\right)$, flattens to rank 2, so $\mathbf{R} \geq 2$; and orbits 2, 3, and 5 — $\left(\begin{smallmatrix} 0 & a_{01} \\ a_{01} & 0 \end{smallmatrix}\right)$, $\left(\begin{smallmatrix} 0 & 0 \\ a_{10} & a_{11} \end{smallmatrix}\right)$, $\left(\begin{smallmatrix} 0 & a_{01} \\ a_{10} & 0 \end{smallmatrix}\right)$ — each flatten to rank 4, so $\mathbf{R} \geq 4$.

Orbit 4, $A = \left(\begin{smallmatrix} 0 & a_{01} \\ a_{01} & a_{11} \end{smallmatrix}\right)$ (forced product). Flattening yields only 4. The forced-product technique automates a substitution of Hopcroft and Kerr [21]: two of the tensor's slices are single products that any optimal decomposition may be assumed to compute literally, which uses up 2 of its terms; stripping those two terms leaves a residual whose every $\mathbb{F}_2$-completion flattens to rank at least 4. Hence $\mathbf{R} \geq 2 + 4 = 6$.

Orbit 6, $A = \left(\begin{smallmatrix} a_{00} & a_{01} \\ a_{01} & a_{00}+a_{01} \end{smallmatrix}\right)$ (substitution). Flattening again gives only 4, so we substitute. Fix an optimal decomposition; its A-components are nonzero linear forms in a_{00}, a_{01}, of which there are just three (a_{00}, a_{01}, $a_{00} + a_{01}$). With at least 4 terms, by pigeonhole one form repeats, and setting it to zero kills at least 2 terms and lands in Orbit 2 (bound 4). Hence $\mathbf{R} \geq 2 + 4 = 6$.

Orbits 7, 8 (degenerate reduction). Adding one more constraint shrinks a subspace to one already settled, whose bound is then inherited. For orbit 7, $A = \left(\begin{smallmatrix} 0 & a_{01} \\ a_{10} & a_{11} \end{smallmatrix}\right)$, adding $a_{01} + a_{10} = 0$ lands in orbit 4; for orbit 8, $A = \left(\begin{smallmatrix} a_{00} & a_{01} \\ a_{01} & a_{11} \end{smallmatrix}\right)$, adding $a_{00} = 0$ lands in orbit 4. Either way $\mathbf{R} \geq 6$.

Orbit 9, $A = \left(\begin{smallmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{smallmatrix}\right)$ (substitution). This is the unconstrained tensor, $\langle 2,2,2\rangle$ itself. Fix an optimal decomposition and take any of its A-components — a nonzero linear form in the four a_{ij}.

Setting it to zero kills at least one term and lands in Orbit 7 or Orbit 8, each of bound 6. Since this holds for *every* nonzero form, $\mathbf{R} \geq 1 + 6$, that is, $\mathbf{R}_{\mathbb{F}_2}(\langle 2, 2, 2\rangle) \geq 7$.

Appendix D works this example in full.

3 The framework

3.1 Tensor rank

Fix a finite field $\mathbb{F}_q$ with $q = p^m$. A multiplication problem is given by an order-3 tensor

$$T = \sum_{i,j,k} t_{ijk}\, a_i \otimes b_j \otimes c_k \;\in\; A \otimes B \otimes C,$$

where $A = \mathbb{F}_q^{N_A}$, $B = \mathbb{F}_q^{N_B}$, $C = \mathbb{F}_q^{N_C}$ have bases a_i, b_j, c_k and the structure constants $t_{ijk} \in \mathbb{F}_q$ encode the bilinear map. The *rank* $\mathbf{R}(T)$ is the least r such that $T = \sum_{\lambda=1}^{r} u_\lambda \otimes v_\lambda \otimes w_\lambda$ with $u_\lambda \in A$, $v_\lambda \in B$, $w_\lambda \in C$; this is exactly the bilinear complexity of the underlying map. We write $\mathbf{R}_{\mathbb{F}_q}(T)$ when emphasizing the field $\mathbb{F}_q$, and $\mathbf{R}(T)$ when it is clear from context.

3.2 Constraints

We attack $\mathbf{R}(T)$ by constraining the first argument A to a subspace. A *constraint* is a linear functional on A, i.e. a vector in the dual $A^* = \mathbb{F}_q^{N_A}$; a set of constraints cuts A down to the subspace on which all of them vanish, and substituting that subspace into T yields a smaller tensor T_S with $\mathbf{R}(T) \geq \mathbf{R}(T_S)$ for any subspace S obtained by adding constraints. A set of d independent constraints spans a d-dimensional subspace of A^*, and two constraint sets that span the same subspace define the same T_S. We therefore represent a constraint subspace by its *reduced row echelon form* (RREF), which is a unique key for the subspace.

3.3 Symmetries and the query/store factorization

The leverage in our method comes from symmetries that act on A and preserve $\mathbf{R}(T)$. Abstractly, let G be a group acting on A^* by semilinear bijections: each g is additive with $g(\lambda u) = \phi_g(\lambda)\, g(u)$ for some field automorphism $\phi_g \in \mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p)$, so both the $\mathbb{F}_q$-linear sandwich and multiplication maps ($\phi_g = \mathrm{id}$) and the Galois symmetries (ϕ_g a Frobenius $a \mapsto a^p$) are allowed. We require that for every $g \in G$ there is a rank-preserving symmetry of T whose action on the first factor is g. Since a semilinear bijection sends $\mathbb{F}_q$-subspaces to $\mathbb{F}_q$-subspaces, G permutes constraint subspaces and maps each T_S to an isomorphic tensor, so $\mathbf{R}(T_S)$ is constant on each G-orbit of subspaces. Our dynamic program needs only one representative per orbit.

The two facts the dynamic program rests on are the following.

Lemma 1 (Monotonicity and orbit invariance)**.** *If $T_{S'}$ is obtained from T_S by imposing additional constraints on the first argument, then $\mathbf{R}(T_S) \geq \mathbf{R}(T_{S'})$. Moreover, for any rank-preserving first-argument symmetry $g \in G$ we have $\mathbf{R}(T_S) = \mathbf{R}(T_{g(S)})$, so $\mathbf{R}(T_S)$ depends only on the G-orbit of the constraint subspace.*

Proof. A bilinear algorithm with r products that computes the map of T_S also computes its restriction to any smaller first-argument subspace with the same r products, so imposing further constraints cannot raise the rank; this is monotonicity. For invariance, the rank-preserving symmetry whose first-factor action is g carries T_S to $T_{g(S)}$ by an invertible change of variables, which leaves the rank unchanged. □

The implementation never materializes G as a list. Instead it factors G into two subsets, a *query* set K and a *store* set Σ, whose product set $\mathrm{K}^{-1} \cdot \Sigma$ covers G so that every orbit member is reached; neither need be a subgroup or be closed under inversion. This is the meet-in-the-middle, or square-root, trick. To test whether a query subspace g lies in the orbit of a stored representative g', we precompute and hash the RREF of $\sigma(g')$ for every $\sigma \in \Sigma$ (the *store images*), and at query time probe the hash with the RREF of $\kappa(g)$ for every $\kappa \in \mathrm{K}$. A hit $\kappa(g) \equiv \sigma(g')$ certifies g and g' as G-equivalent and yields the witness pair (κ, σ), from which the canonical form is recovered as

$$g' \;\equiv\; \sigma^{-1}\big(\kappa(g)\big). \tag{1}$$

Storage grows by a factor $|\Sigma|$ and a query costs $|\mathrm{K}|$ probes, with $|\mathrm{K}| \cdot |\Sigma| \gtrsim |G|$; choosing $|\mathrm{K}| \approx |\Sigma| \approx \sqrt{|G|}$ trades square-root space for square-root time.

4 Symmetries of the problem families

The framework is fixed once we name, for each problem, the tensor T and the factorization (K, Σ) of its first-argument symmetry group.

4.1 Matrix multiplication

The tensor for multiplying an $l \times m$ matrix X by an $m \times n$ matrix Y, with product $Z^\top$, is

$$\langle l, m, n \rangle = \sum_{i=0}^{l-1} \sum_{j=0}^{m-1} \sum_{k=0}^{n-1} x_{ij} \otimes y_{jk} \otimes z_{ki},$$

so $A = \mathbb{F}_q^{l \times m}$. For invertible $P \in \mathrm{GL}_l$, $Q \in \mathrm{GL}_m$, $R \in \mathrm{GL}_n$ the substitution $X \mapsto PXQ^{-1}$, $Y \mapsto QYR^{-1}$, $Z \mapsto RZP^{-1}$ preserves the tensor (the *sandwich* symmetry). The rank is also invariant under cyclically permuting the three factors, $\mathbf{R}(\langle l, m, n \rangle) = \mathbf{R}(\langle m, n, l \rangle)$ (the *cyclic* symmetry); for a square format $l = m = n$ this combines with matrix transposition into an order-two symmetry $X \mapsto X^\top$, $Y \mapsto Z^\top$, $Z \mapsto Y^\top$ (the *transpose* symmetry). Projected onto the first factor, the symmetry group acting on the matrix space $A = \mathbb{F}_q^{l \times m}$ is

$$G \;=\; (\mathrm{GL}_l(\mathbb{F}_q) \times \mathrm{GL}_m(\mathbb{F}_q)) \rtimes C_2,$$

with (P, Q) acting by $X \mapsto PXQ^{-1}$ and the C_2 by transposition $X \mapsto X^\top$ (present only when $l = m = n$). The meet-in-the-middle splits G by side: the store set is the right multiplications $\Sigma = \{X \mapsto XR^{-1} : R \in \mathrm{GL}_m(\mathbb{F}_q)\}$ and the query set is the left multiplications $\mathrm{K} = \{X \mapsto LX : L \in \mathrm{GL}_l(\mathbb{F}_q)\}$, extended by the transpose when $l = m = n$. Scalar matrices act trivially on subspaces, so each side is taken modulo the center, and both then have order about $\sqrt{|G|}$. Equivalence of constraint subspaces under the subgroup without transpose is exactly the tensor-isomorphism problem on $l \times m \times d$ tensors, which sits above graph and code equivalence in the isomorphism hierarchy [19, 20].

4.2 Full polynomial multiplication

Multiplying two polynomials of degree $< N$ in $\mathbb{F}_q[x]$ gives

$$\mathrm{P}_N = \sum_{i,j=0}^{N-1} a_i \otimes b_j \otimes c_{i+j}, \qquad N_A = N_B = N,\ N_C = 2N-1,$$

the ordinary convolution. Its first-argument symmetries are the action on degree-$(N-1)$ binary forms of the projective semilinear group

$$G_{\text{full}} = \mathrm{P\Gamma L}_2(\mathbb{F}_q) = \mathrm{PGL}_2(\mathbb{F}_q) \rtimes \mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p),$$

generated by the substitutions $x \mapsto x + c$, the reversal $x \mapsto 1/x$ (which swaps $a_i \leftrightarrow a_{N-1-i}$), the scalings $x \mapsto gx$, and the Frobenius $a \mapsto a^p$. We factor it as $\Sigma = \mathrm{PGL}_2(\mathbb{F}_q)$ (the store side, of size $q(q^2-1)$) and $\mathrm{K} = \mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p)$ (the query side, of size m).

4.3 Polynomial multiplication modulo $x^N - \gamma$

Fix a scalar $\gamma \in \mathbb{F}_q$ and let $R = \mathbb{F}_q[x]/(x^N - \gamma)$. Multiplication in R reduces the convolution of a and b by the rule $x^N \equiv \gamma$,

$$\sum_{i,j=0}^{N-1} a_i \otimes b_j \otimes w_{ij}, \qquad w_{ij} = \begin{cases} c_{i+j}, & i+j < N, \\ \gamma\, c_{i+j-N}, & i+j \geq N, \end{cases} \qquad N_A = N_B = N_C = N.$$

Three families specialize this tensor: *cyclic* convolution C_N for $\gamma = 1$, the *truncated* (or short) product T_N for $\gamma = 0$ — the local algebra $\mathbb{F}_q[x]/(x^N)$, in which the overflow terms vanish and x is nilpotent — and the *negacyclic* product C_N^- for $\gamma = -1$. Over characteristic 2, $-1 = 1$, so $\mathrm{C}_N^- = \mathrm{C}_N$.

The same three kinds of symmetry act on $A = R$ for every γ: multiplication by a unit $p \in R^*$ (the shifts x^r are the special case $p = x^r$, valid for $\gamma \neq 0$), the $\mathbb{F}_q$-algebra automorphisms $\mathrm{Aut}(R)$, and the Galois group of $\mathbb{F}_q/\mathbb{F}_p$. The base-field scalars $\mathbb{F}_q^*$ act trivially on subspaces, so the group acting on constraint subspaces is

$$G = \big(R^* \rtimes \mathrm{Aut}(R) \rtimes \mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p)\big)/\mathbb{F}_q^*,$$

with store side $\Sigma = R^*/\mathbb{F}_q^*$ and query side $\mathrm{K} = \mathrm{Aut}(R) \rtimes \mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p)$ — the identical factorization in all three cases. Only the ring structure, and hence the sizes of the two sides, depends on γ. For $\gamma = \pm 1$ the Chinese remainder theorem splits R into field factors — for $\gamma = 1$ indexed by the q-cyclotomic cosets of $\mathbb{Z}/N$, for $\gamma = -1$ by the cosets of the odd residues modulo $2N$ — so R^* is typically far larger than the N shifts, which is precisely the extra symmetry the framework exploits. For $\gamma = 0$ the algebra is local: a polynomial is a unit iff its constant term is nonzero, so $|\Sigma| = q^{N-1}$, and an automorphism sends x to any nilpotent generator $y_1 x + \cdots + y_{N-1}x^{N-1}$ with $y_1 \neq 0$, giving $|\mathrm{Aut}(R)| = (q-1)q^{N-2}$. Table 2 contrasts the symmetry structures; the three quotient families share one row, differing only in γ.

5 Orbit enumeration

The first stage enumerates exactly one canonical representative per orbit of constraint subspaces, for every dimension $d = 0, 1, \ldots, N_A$. Subspaces are built up one constraint at a time and deduplicated with the same meet-in-the-middle trick.

problem	store Σ	query K
matrix-mult $\langle l, m, n\rangle$	GL_m (right mult.)	GL_l (left mult.), and transpose ($l = m = n$)
full-poly-mult	$\mathrm{PGL}_2(\mathbb{F}_q)$	$\mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p)$
poly-mult $\mathbb{F}_q[x]/(x^N-\gamma)$	$R^*/\mathbb{F}_q^*$	$\mathrm{Aut}(R) \rtimes \mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p)$

Table 2: First-argument symmetry factorizations; the last row covers the cyclic ($\gamma = 1$), truncated ($\gamma = 0$), and negacyclic ($\gamma = -1$) products. In every family the query elements are indexed so that the i-th element is i, which is why a certificate can name a symmetry by a single integer.

Representatives are generated dimension by dimension, in lexicographic order of their RREF. A dimension-d subspace is formed by appending to a dimension-$(d-1)$ representative one new constraint whose leading pivot lies strictly above the existing pivots; candidates with a nonzero entry at an existing pivot are skipped because Gauss–Jordan elimination would reduce them against the current basis. Because candidates are visited in global lexicographic order, the first time an orbit is seen its representative is automatically the lexicographically least, so no separate minimality test is needed.

Deduplication uses a hash set of RREF keys. For each kept representative c we insert the RREF of every store image $\sigma(c)$, $\sigma \in \Sigma$; for each new candidate q we probe with the RREF of every query image $\kappa(q)$, $\kappa \in \mathrm{K}$, and discard q if any probe hits, since a hit means $q \equiv (\kappa^{-1}\sigma)(c)$ lies in c's orbit. Algorithm 1, in Appendix C, states the procedure.

6 Rank lower-bound techniques

The dynamic program sweeps subspace dimension from N_A down to 0, processing all orbits of each dimension in parallel and inserting their freshly computed bounds into the orbit map before moving to the next (smaller) dimension. This ordering is sound because the only techniques that look anything up — degenerate reduction and backtracking — query *strictly larger* dimensions, already settled. Each orbit tries four techniques in order — flatten, degenerate reduction, forced product, backtracking — stopping once one meets the goal.

Flatten. Compute the three flattening matrix ranks of the constrained tensor and take the maximum. This is the base case and needs no other orbit.

Degenerate reduction. Add one more independent constraint to the current subspace, producing a strictly smaller subspace whose orbit was processed at the previous (larger) dimension; by Lemma 1 its bound is a valid bound for the current orbit. We scan candidate constraints, look up each enlarged subspace in the orbit map, and keep the best. The proof records the extra constraint together with the witness pair from Equation (1).

Forced product. This automates the Hopcroft–Kerr substitution [21], which rests on the following lemma.

Lemma 2 (Hopcroft and Kerr [21, Lemma 2])**.** *Let $f_0, \ldots, f_{t-1}$ be bilinear forms of which $f_0, \ldots, f_{s-1}$ are linearly independent and each is a single product (a form of rank 1). If the whole set can be*

computed with r multiplications, then it can be computed with r multiplications s of which are exactly $f_0, \ldots, f_{s-1}$.

Proof. Take any bilinear algorithm with products $p_0, \ldots, p_{r-1}$ computing the f_i, each a fixed $\mathbb{F}_q$-linear combination of the products. Exchange products for the f_i one at a time, maintaining the invariant that the current r products are $f_0, \ldots, f_{i-1}$ together with $r-i$ of the original p_j, spanning all the f_i. At step i, since f_i is independent of $f_0, \ldots, f_{i-1}$, its expression in the current products gives a nonzero coefficient to some remaining *original* product p_j; then p_j lies in the span of f_i and the other current products, so swapping p_j for f_i — itself a legal product, having rank 1 — preserves the invariant. After s steps the algorithm has r products, s of which are exactly $f_0, \ldots, f_{s-1}$. □

Grouping the tensor along the C factor into t slices, those whose $A \otimes B$ part has matrix rank 1 are single products; we greedily take a maximal independent set of s of them and, by Lemma 2, assume an optimal decomposition computes them literally. Stripping these s terms leaves $s(t-s)$ unknown field coefficients in the residual; we enumerate all $q^{s(t-s)}$ assignments, flatten each, and take the minimum, so the bound is $s + \min$. We repeat for all three cyclic orientations and keep the best. Since the minimum must range over *every* assignment for soundness, we skip the technique when $q^{s(t-s)}$ is too large to enumerate. The proof records only which orientation won. Note that Lemma 2 holds over any field; it is the exhaustive enumeration of the $q^{s(t-s)}$ residuals that makes the technique *effective* only over a small finite field.

Substitution with backtracking. The substitution method of Pan [28] is a powerful technique in bilinear complexity lower bound proofs

Lemma 3 (Substitution)**.** *Fix an optimal decomposition $T_S = \sum_\lambda u_\lambda \otimes v_\lambda \otimes w_\lambda$ with $\mathbf{R}(T_S)$ terms, each A-component u_λ a nonzero linear form in the surviving A-variables. Suppose that repeatedly setting a linear form to zero — each step imposing that form as a new constraint and discarding the terms it annihilates — removes δ terms in total and reaches the more-constrained tensor $T_{S'}$. Then $\mathbf{R}(T_S) \geq \delta + \mathbf{R}(T_{S'})$.*

Proof. Setting a linear form to zero constrains the first argument to the hyperplane it defines; the terms whose A-component that constraint annihilates drop out, and the surviving terms remain a valid decomposition of the constrained tensor. So a single step that discards d terms gives $\mathbf{R}(T_S) \geq d + \mathbf{R}(T_{S''})$ for the one-larger subspace S''. Iterating along the chain and summing the discarded terms yields the claim. □

So if a chain of substitutions removing δ terms lands in an orbit of already-certified bound b with $\delta + b \geq$ target, then $\mathbf{R}(T_S) \geq$ target. The technique searches for such a chain by DFS over canonical A-components — the nonzero linear forms supported away from the current pivots, one per coset. The DFS depth is capped at the known lower bound given by the previous techniques.

Soundness rests on a completeness invariant: the case analysis over the unknown A-components must be exhaustive, so the technique certifies the target only if *every* branch of the DFS — one per canonical nonzero form at each level — reaches it. Several optimizations make the search scale without weakening this invariant: the DFS branches are run in parallel; a thread-local cache memoizes the bound of each enlarged subspace, halving itself stochastically when it grows too large; the target bound is raised one unit at a time so that easy branches prune early; and a step limit aborts large branches — an aborted branch aborts the technique for the whole orbit, trading away

completeness of the *search* but never soundness of a reported bound, and the dynamic program falls back to the other techniques. On success, each branch contributes one record to the proof: the depth, the subset mask of substituted forms, and the witness pair identifying the orbit it reduced to.

7 Proof certificates and verification

Every bound is emitted as a certificate that a verifier rechecks; the verifier is simpler than the search, and runs faster with less memory. For each orbit the certificate records its canonical constraint subspace, the certified lower bound, and a proof that is exactly one of the four techniques (the unconstrained orbit's bound is "the" bound for the problem). Each proof stores only what the verifier cannot cheaply recompute:

- **Flatten:** nothing; the verifier recomputes the flattening rank.
- **Forced product:** the winning cyclic orientation, an integer in $\{0, 1, 2\}$.
- **Degenerate:** the extra constraint, together with the witness pair (κ, σ) of Equation (1) that identifies the enlarged subspace's orbit.
- **Backtracking:** for each DFS leaf, its depth, the subset of substituted forms, and the witness pair of the orbit it reduced to. The verifier replays the case analysis and confirms that the recorded leaves cover *every* canonical nonzero form at each level.

The verifier replays the same sweep from dimension N_A down to 0, so each orbit is checked only against strictly larger orbits already confirmed. It rechecks flatten and forced product by recomputation, and degenerate and backtracking by reconstructing the canonical form of every claimed reduction from its witness via Equation (1) and confirming the looked-up bound supports the claim. A witness that misses a known orbit, a recomputed bound that falls short, or a miscounted trace aborts verification. Soundness follows by induction on decreasing dimension: at dimension N_A only flatten and forced product apply, and every smaller-dimension step reduces to an orbit verified earlier.

8 Conclusion and open problems

We have given a single automated framework for bilinear-complexity lower bounds over finite fields, parameterized only by a tensor and a symmetry group, and used it to improve the long-standing bound on $\langle 3, 3, 3\rangle$ over $\mathbb{F}_2$ to 20, to settle three more matrix formats, and to obtain eighteen new lower bounds for polynomial multiplication across the full, cyclic, truncated, and negacyclic products. Every bound comes with a certificate that an auditable verifier rechecks, typically faster than the search. The source code, verifier, and all certificates are available at `https://github.com/wcgbg/tensor-rank-lower-bound`.

Several questions remain. The exact rank of $\langle 3, 3, 3\rangle$ over $\mathbb{F}_2$ is still open in $[20, 23]$, and scaling to $\langle 4, 4, 4\rangle$ is blocked by the sheer number of constraint orbits, needing stronger symmetry reduction; the polynomial bounds should extend to larger N with more compute. Natural next steps are extension-field multiplication in $\mathbb{F}_{q^n}$ over $\mathbb{F}_q$ ($f \cdot g \bmod p(x)$ for an irreducible p of degree n), and genuinely new bounds beyond $\mathbb{F}_2$ and $\mathbb{F}_3$, to which the framework applies verbatim.

Acknowledgements

Thanks to Youming Qiao for helpful discussions on the tensor isomorphism problem.

References

[1] Alexander Alder and Volker Strassen. On the algorithmic complexity of associative algebras. *Theoretical Computer Science*, 15(2):201–211, 1981.

[2] Amir Averbuch, Zvi Galil, and Shmuel Winograd. Classification of all the minimal bilinear algorithms for computing the coefficients of the product of two polynomials modulo a polynomial, part i: the algebra $g[u]/\langle q(u)^l\rangle$, $l > 1$. *Theoretical Computer Science*, 58(1):17–56, 1988.

[3] Amir Averbuch, Zvi Galil, and Shmuel Winograd. Classification of all the minimal bilinear algorithms for computing the coefficients of the product of two polynomials modulo a polynomial, part ii: the algebra $g[u]/\langle u^n\rangle$. *Theoretical Computer Science*, 86(2):143–203, 1991.

[4] Razvan Barbulescu, Jérémie Detrey, Nicolas Estibals, and Paul Zimmermann. Finding optimal formulae for bilinear maps. In *Arithmetic of Finite Fields (WAIFI 2012)*, volume 7369 of *Lecture Notes in Computer Science*, pages 168–186. Springer, 2012.

[5] Stefan Beuchert. Independent Verification of the Wang $\mathbb{F}_2$-Certificate for $\mathbf{R}(\langle 3,3,3\rangle) \geq 20$. Zenodo, `https://doi.org/10.5281/zenodo.20752782`, 2026.

[6] Markus Bläser. A $5/2n^2$-lower bound for the multiplicative complexity of $n \times n$-matrix multiplication. In *Proceedings of the 40th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 45–50. IEEE, 1999.

[7] Markus Bläser. Lower bounds for the multiplicative complexity of matrix multiplication. *Computational Complexity*, 8(3):203–226, 1999.

[8] Markus Bläser. On the complexity of the multiplication of matrices of small formats. *Journal of Complexity*, 19(1):43–60, 2003.

[9] Markus Bläser. A complete characterization of the algebras of minimal bilinear complexity. *SIAM Journal on Computing*, 34(2):277–298, 2005.

[10] Nader H. Bshouty. A lower bound for matrix multiplication. *SIAM Journal on Computing*, 18(4):759–765, 1989.

[11] Murat Cenk and Ferruh Özbudak. On multiplication in finite fields. *Journal of Complexity*, 26(2):172–186, 2010.

[12] Murat Cenk and Ferruh Özbudak. Multiplication of polynomials modulo x^n. *Theoretical Computer Science*, 412(29):3451–3462, 2011.

[13] David V. Chudnovsky and Gregory V. Chudnovsky. Algebraic complexities and algebraic curves over finite fields. *Journal of Complexity*, 4(4):285–316, 1988.

[14] Hans F. de Groote. Characterization of division algebras of minimal rank and the structure of their algorithm varieties. *SIAM Journal on Computing*, 12(1):101–117, 1983.

[15] Haining Fan and M Anwar Hasan. Comments on "Five, six, and seven-term Karatsuba-like formulae". *IEEE Transactions on Computers*, 56(5):716–717, 2007.

[16] Charles M. Fiduccia and Yechezkel Zalcstein. Algebras having linear multiplicative complexities. *Journal of the ACM*, 24(2):311–331, 1977.

[17] Markus Grassl. Bounds on the minimum distance of linear codes and quantum codes. Online, `http://www.codetables.de`, 2007.

[18] James H. Griesmer. A bound for error-correcting codes. *IBM Journal of Research and Development*, 4(5):532–542, 1960.

[19] Joshua A. Grochow and Youming Qiao. On the complexity of isomorphism problems for tensors, groups, and polynomials i: tensor isomorphism-completeness. *SIAM Journal on Computing*, 52(2):568–617, 2023.

[20] Joshua A. Grochow and Youming Qiao. On the complexity of isomorphism problems for tensors, groups, and polynomials iv: linear-length reductions and their applications. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, pages 766–776, 2025.

[21] John E. Hopcroft and Leslie R. Kerr. On minimizing the number of multiplications necessary for matrix multiplication. *SIAM Journal on Applied Mathematics*, 20(1):30–36, 1971.

[22] Michael Kaminski and Nader H. Bshouty. Multiplicative complexity of polynomial multiplication over finite fields. *Journal of the ACM*, 36(1):150–170, 1989.

[23] Manuel Kauers and Jakob Moosbauer. Flip graphs for matrix multiplication. In *Proceedings of the 2023 International Symposium on Symbolic and Algebraic Computation*, pages 381–388, 2023.

[24] Julian D. Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. *Bulletin of the American Mathematical Society*, 82(1):126–128, 1976.

[25] Abraham Lempel, Gadiel Seroussi, and Shmuel Winograd. On the complexity of multiplication in finite fields. *Theoretical Computer Science*, 22(3):285–296, 1983.

[26] Peter L. Montgomery. Five, six, and seven-term Karatsuba-like formulae. *IEEE Transactions on Computers*, 54(3):362–369, 2005.

[27] Salvatore D. Morgera. Multiplicative complexity of bilinear algorithms for cyclic convolution over finite fields. *Multidimensional Systems and Signal Processing*, 1(1):99–111, 1990.

[28] V Ya Pan. Methods of computing values of polynomials. *Russian Mathematical Surveys*, 21(1):105–136, 1966.

[29] Alexey V. Smirnov. The bilinear complexity and practical algorithms for matrix multiplication. *Computational Mathematics and Mathematical Physics*, 53(12):1781–1795, 2013.

[30] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.

[31] Andrei L. Toom. The complexity of a scheme of functional elements realizing the multiplication of integers. *Soviet Mathematics Doklady*, 3:714–716, 1963.

[32] Meghanad D. Wagh and Salvatore D. Morgera. A new structured design method for convolutions over finite fields, part i. *IEEE Transactions on Information Theory*, 29(4):583–595, 1983.

[33] Shmuel Winograd. On multiplication of 2×2 matrices. *Linear Algebra and its Applications*, 4(4):381–388, 1971.

[34] Shmuel Winograd. Some bilinear forms whose multiplicative complexity depends on the field of constants. *Mathematical Systems Theory*, 10(2):169–180, 1977.

A Complete bound tables by family

Tables 3–7 give the full lower- and upper-bound picture for each problem family. The boxed entries are the new lower bounds proved in this paper and collected in Table 1. Over characteristic 2 the negacyclic product coincides with cyclic convolution ($x^N + 1 = x^N - 1$), so Table 7 lists only $\mathbb{F}_3$.

In five entries the previously known lower bound exceeds ours by one: cyclic convolution at $N = 5$ over both $\mathbb{F}_2$ and $\mathbb{F}_3$ and at $N = 9$ over $\mathbb{F}_2$ (Table 5), the truncated product at $N = 4$ over $\mathbb{F}_3$ (Table 6), and the negacyclic product at $N = 5$ over $\mathbb{F}_3$ (Table 7). These entries share a pattern: the previous bound is tight (it meets the best known upper bound, so the rank is known exactly), and it comes from the algebra-specific minimal-rank theory of Bläser [9] and of Averbuch, Galil, and Winograd [3], which exploits the multiplicative structure of the quotient algebra. Our substitution-based techniques do not subsume those arguments, and in each of these cases they stop one short of the known rank; the "our LB" column always reports what our framework proves, not the stronger inherited bound.

The tables also serve as a global sanity check on the method: in no entry does a bound we prove exceed the best known upper bound, and in many small cases it matches the exact rank already known. A soundness bug inflating a bound would, across the dozens of tabulated cases, likely have produced a lower bound exceeding some known upper bound, or a lower bound that is much lower than the best known lower bound.

format	prev LB	our LB	prev UB
$\langle 2,2,2\rangle$	7 [33]	7	7 [30]
$\langle 2,2,3\rangle$	11 [21]	11	11 ($\langle 2,2,2:7\rangle + \langle 2,2,1:4\rangle$)
$\langle 2,2,4\rangle$	14 [21]	14	14 ($\langle 2,2,2:7\rangle + \langle 2,2,2:7\rangle$)
$\langle 2,3,3\rangle$	15 [21]	15	15 [21]
$\langle 2,3,4\rangle$	18 [7], 19? [21]	$\boxed{19}$	20 [21]
$\langle 3,3,3\rangle$	19 [8]	$\boxed{20}$	23 [24]
$\langle 3,3,4\rangle$	24 [8]	$\boxed{25}$	29 [29]
$\langle 3,4,4\rangle$	28 [7]	$\boxed{29}$	38 [29]

Table 3: Matrix multiplication tensor rank over $\mathbb{F}_2$; boxed entries are new.

	over $\mathbb{F}_2$			over $\mathbb{F}_3$		
N	prev LB	our LB	prev UB	prev LB	our LB	prev UB
1	1	1	1	1	1	1
2	3 [16]	3	3 [31]	3 [16]	3	3 [31]
3	6 [25, 13, 18]	6	6 [22]	6 [25, 13, 17]	6	6 [22]
4	9 [22]	9	9 [22]	9 [22]	9	9 [22]
5	13 [25, 13, 17]	13	13 [26]	12 [22]	12	12 [22]
6	15 [25, 13, 17]	16	17 [26]	12 [25, 13, 18]	14	15 [34]
7	18 [25, 13, 17]	19	22 [26]			
8	20 [25, 13, 17]	21	26 [34, 15]			

Table 4: Full product of two degree-$(N-1)$ polynomials; boxed entries are new.

	over $\mathbb{F}_2$			over $\mathbb{F}_3$		
N	prev LB	our LB	prev UB	prev LB	our LB	prev UB
1	1	1	1	1	1	1
2	3 [34]	3	3 [27]	2 [34]	2	2 [32]
3	4 [34]	4	4 [32]	5 [34]	5	5 [27]
4	8 [9, 4]	8	8 [27, 12]	5 [34]	5	5 [32]
5	10 [13, 9]	9	10 [32]	10 [9, 4]	9	10 [32]
6	11 [2, 9]	11	12 [27]	10 [34]	10	10 [27]
7	12 [14, 9]	13	13 [32]	13 [14, 9]	13	16 [27, 11]
8	16 [3, 9]	19	22 [27, 12]	11 [34]	11	11 [32]
9	19 [13, 9]	18	19 [27]	18 [3, 9]	19	27 [27, 12]
10	19 [2, 9]	22	29 [27]			

Table 5: Cyclic convolution modulo $x^N - 1$; boxed entries are new.

B Computational results

Table 8 reports the search and verification details for each new bound. The bounds themselves and their comparison to prior work appear in Tables 3–7 of Appendix A.

C Orbit-enumeration algorithm

Algorithm 1 spells out the orbit enumerator of Section 5: each dimension-d representative is built by extending a dimension-$(d-1)$ one with a new highest-pivot constraint, and the inner duplicate test is the meet-in-the-middle probe of that section. The *visited* set is cleared at the start of each dimension, since the subspace dimension is an invariant of the group action.

	over $\mathbb{F}_2$			over $\mathbb{F}_3$		
N	prev LB	our LB	prev UB	prev LB	our LB	prev UB
1	1	1	1	1	1	1
2	3 [34, 1]	3	3 [31]	3 [34, 1]	3	3 [31]
3	5 [34, 1]	5	5 [3, 9]	5 [34, 1]	5	5 [3, 9]
4	8 [3, 9]	8	8 [12]	8 [3, 9]	7	8 [12]
5	10 [3, 9]	10	11 [12]	10 [3, 9]	10	10 [4]
6	12 [3, 9]	$\boxed{13}$	14 [12]	12 [3, 9]	12	14 [12]
7	14 [3, 9]	$\boxed{16}$	18 [12]	14 [3, 9]	14	18 [12]
8	16 [3, 9]	$\boxed{19}$	22 [12]	16 [3, 9]	$\boxed{17}$	22 [12]
9	18 [3, 9]	$\boxed{21}$	27 [12]*	18 [3, 9]	$\boxed{19}$	27 [34]*
10	20 [3, 9]	$\boxed{24}$	31 [12]	20 [3, 9]	$\boxed{21}$	31 [34]
11	22 [3, 9]	$\boxed{27}$	36 [12]			

* We obtain a new bilinear complexity upper bound of 26 for multiplication over $\mathbb{Z}[x]/x^9$ by flip-graph search on $\mathbb{F}_2$ [23] and lifting to $\mathbb{Z}$.
$(5a1-6a2+2a3+a5-a6)\otimes(3b0+2b1+b2)\otimes(-c6-c7-c8)+a0\otimes b0\otimes(c0-c3-c5-c8)+(a1-a2)\otimes(3b0+2b1+b2)\otimes(-c2-c3-c4-c6)+(a0-a1)\otimes(b0+b1+3b2+3b3+b4+b6+b7)\otimes(-c7-c8)+(2a0-a1)\otimes(b0-b5+b7)\otimes c7+(3a0-2a1+a2-2a3+a4)\otimes(4b0+3b1+b2+b3+b4)\otimes c4+(a2-4a3+2a4-2a5+3a6-a7)\otimes(b0+b1)\otimes c7+(a1-a2)\otimes(3b0+b1-b2-3b3-3b4-2b5-b6)\otimes(c6+c7+c8)+(a2-a3+a4-2a5+a6)\otimes(2b0+2b1+b2)\otimes(-c6-c7)+(a0-a2+a3)\otimes(b0+b1+b4+b5)\otimes(-c3+c5+c8)+(2a0-a1)\otimes(b0+b1)\otimes(c1-c2-c3-c4-c5)+(a0-2a1+2a2-a3)\otimes(6b0+4b1+2b2+b3+b4+b5)\otimes(-c3-c6-c7-c8)+(a0+2a1-a2-2a3+2a4-a5)\otimes(b0+b1+b2)\otimes c6+a0\otimes(b1+b2+b3+b4+b6+b7+b8)\otimes c8+(a0-a1+a2-2a3+a4)\otimes(5b0+4b1+b2+b3+b4)\otimes(-c4-c7)+(a0-a1-a2-a3+a4)\otimes(5b0+3b1+2b2+2b3+b4)\otimes(-c4-c5)+(a0+a1-a2)\otimes(3b0+2b1+b2+b3+b4+b5+b6)\otimes(c6+c7)+(6a0-3a1-2a2-a3+2a6-a7)\otimes(2b0+b1)\otimes(-c7-c8)+(6a0-a1+a3-a4+a6-2a7+a8)\otimes b0\otimes c8+(a2-a3)\otimes(b2-b4-b5)\otimes(c3-c5-c6-c8)+(a2+a3-a4)\otimes(7b0+4b1+2b2+2b3+b4)\otimes(-c4-c5-c7-c8)+(2a0-2a1+2a2-a3)\otimes(4b0+3b1+b2+b3+b4+b5)\otimes(c3+c7)+(a0+a1-a2)\otimes(2b0+2b1+b2)\otimes(c2+c7+c8)+(a0+a3-2a4+a5)\otimes(2b0+2b1+b2+b3)\otimes(-c5-c7)+(3a0-2a1+a2+a3-2a4+a5)\otimes(3b0+2b1+b2+b3)\otimes(c5+c7+c8)+(a0-a1)\otimes(2b0+b1)\otimes(-c1+c3+c6+c7+c8)$

Table 6: Truncated product modulo x^N; boxed entries are new lower bounds.

	over $\mathbb{F}_3$		
N	prev LB	our LB	prev UB
1	1	1	1
2	3 [34]	3	3 [31]
3	5 [34]	5	5 [3, 9]
4	6 [34]	6	6 [14]
5	10 [9, 4]	9	10 [11]
6	12 [2, 9]	12	15 [34]
7	13 [14, 9]	13	16 [11]
8	16 [9, 4]	16	18 [11]
9	18 [3, 9]	$\boxed{19}$	27 [12]

Table 7: Negacyclic convolution modulo x^N+1; the boxed entry is new. Over $\mathbb{F}_2$, $x^N+1=x^N-1$, so the negacyclic product coincides with cyclic convolution (Table 5).

Orbit counts

Table 9 reports the number of constraint-subspace orbits Algorithm 1 finds for each matrix multiplication format over $\mathbb{F}_2$ and $\mathbb{F}_3$. The count depends on whether the transpose symmetry is in

result	machine[†]	search time	search cost	cert. size	verify time	verify cost
$\mathbf{R}_{\mathbb{F}_2}(\langle 2,3,4\rangle) \geq 19^{\ddagger}$	MacBook Air	2 hrs	–	–	2 hrs	–
$\mathbf{R}_{\mathbb{F}_2}(\langle 3,3,3\rangle) \geq 20$	MacBook Air	40 min	–	32 MiB	3 sec	–
$\mathbf{R}_{\mathbb{F}_2}(\langle 3,3,4\rangle) \geq 25$	c8g.{16,48}xlarge[§]	4.6 days	52 USD	600 MiB	70 min	1 USD
$\mathbf{R}_{\mathbb{F}_2}(\langle 3,4,4\rangle) \geq 29$	c8g.48xlarge	4.9 hrs	5 USD	500 MiB	3.4 hrs	3 USD
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_6) \geq 16$	c8g.48xlarge	11 min	–	–	10 min	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_7) \geq 19$	c8g.48xlarge	3 hrs	3 USD	9 MiB	2.6 hrs	3 USD
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{P}_8) \geq 21$	c8g.48xlarge	6 hrs	6 USD	224 MiB	3 hrs	3 USD
$\mathbf{R}_{\mathbb{F}_3}(\mathrm{P}_6) \geq 14$	c8g.48xlarge	35 min	–	6 MiB	31 min	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_7) \geq 13$	c8g.48xlarge	–	–	–	–	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_8) \geq 19$	c8g.48xlarge	–	–	–	–	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{C}_{10}) \geq 22$	c8g.48xlarge	–	–	8 MiB	–	–
$\mathbf{R}_{\mathbb{F}_3}(\mathrm{C}_9) \geq 19$	c8g.48xlarge	3 min	–	3 MiB	–	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_6) \geq 13$	c8g.48xlarge	–	–	–	–	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_7) \geq 16$	c8g.48xlarge	–	–	–	–	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_8) \geq 19$	c8g.48xlarge	–	–	–	–	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_9) \geq 21$	c8g.48xlarge	–	–	–	–	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_{10}) \geq 24$	c8g.48xlarge	1 min	–	3 MiB	–	–
$\mathbf{R}_{\mathbb{F}_2}(\mathrm{T}_{11}) \geq 27$	c8g.48xlarge	7 min	–	22 MiB	–	–
$\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_8) \geq 17$	c8g.48xlarge	–	–	–	–	–
$\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_9) \geq 19$	c8g.48xlarge	2 min	–	2 MiB	–	–
$\mathbf{R}_{\mathbb{F}_3}(\mathrm{T}_{10}) \geq 21$	c8g.48xlarge	3.4 hrs	3 USD	20 MiB	–	–
$\mathbf{R}_{\mathbb{F}_3}(\mathrm{C}_9^-) \geq 19$	c8g.48xlarge	3 min	–	2 MiB	–	–

– Very small number.
[†] Machines: Apple MacBook Air M4 with 16 GB RAM; AWS c8g.16xlarge Spot Instance, Graviton4 processors, 64 vCPUs, 128 GiB memory; AWS c8g.48xlarge Spot Instance, Graviton4 processors, 192 vCPUs, 384 GiB memory.
[‡] Run on the equivalent $\langle 3,2,4\rangle$ format: composing the transpose symmetry $\mathbf{R}(\langle l,m,n\rangle) = \mathbf{R}(\langle n,m,l\rangle)$ with one cyclic shift $\mathbf{R}(\langle l,m,n\rangle) = \mathbf{R}(\langle m,n,l\rangle)$ of Section 4 gives $\mathbf{R}(\langle 2,3,4\rangle) = \mathbf{R}(\langle 4,3,2\rangle) = \mathbf{R}(\langle 3,2,4\rangle)$. The proof is forced-product-heavy, so CPU verification takes about as long as the search. The verification can be accelerated by an NVIDIA L20 GPU to approximately 1 minute.
[§] Search on c8g.16xlarge. Verification on c8g.48xlarge.

Table 8: Search and verification details for each new lower bound.

force, which happens exactly when the format is square ($l = m = n$); the polynomial families are swept by the same procedure.

Where the exact count is out of reach, an orbit-counting (Burnside) estimate gives its order of magnitude. The number of orbits is at least the number of constraint subspaces divided by the group order; approximating the number of d-dimensional subspaces of $\mathbb{F}_q^{lm}$ and summing over d yields

$$\sum_{d=0}^{lm} \frac{q^{lm\,d}}{|\mathrm{GL}_l(\mathbb{F}_q) \times \mathrm{GL}_m(\mathbb{F}_q) \times \mathrm{GL}_d(\mathbb{F}_q)| \cdot |T|}, \tag{2}$$

where $T = C_2$ when the transpose symmetry is present ($l = m = n$) and $T = C_1$ otherwise, and $|\mathrm{GL}_n(\mathbb{F}_q)| = \prod_{k=0}^{n-1}(q^n - q^k)$. This produces the parenthesized entries of Table 9. Both the

Algorithm 1 Enumerate constraint-subspace orbits

reps[0] ← [∅]
for $d = 1$ **to** N_A **do**
    visited ← ∅
    for each representative $c \in$ *reps*$[d-1]$ (in lex order) **do**
        for each candidate constraint r above all pivots of c (in lex order) **do**
            $q \leftarrow c \cup \{r\}$
            if not SEEN(q) **then**
                append q to *reps*$[d]$
                for $\sigma \in \Sigma$ **do** insert RREF($\sigma(q)$) into *visited*
                end for
            end if
        end for
    end for
end for
return *reps*

function SEEN(q)
    for $\kappa \in$ K **do**
        if RREF($\kappa(q)$) $\in$ *visited* **then return true**
        end if
    end for
    return false
end function

format $l \times m$	transpose?	# orbits over $\mathbb{F}_2$	# orbits over $\mathbb{F}_3$
2×2	yes	10	10
2×2	no	11	11
2×3	no	31	31
2×4	no	86	91
3×3	yes	496	736
3×3	no	710	1046
3×4	no	158,426	$>1.6 \times 10^6$ †
4×4	yes	$>1.6 \times 10^{11}$	$>8.8 \times 10^{15}$

† The exact value is computable but unlikely to improve any matrix bound.

Table 9: Number of constraint-subspace orbits for matrix multiplication, $A = \mathbb{F}_q^{l \times m}$. "transpose?" marks whether the order-two transpose symmetry is included; it applies only to square formats $l = m = n$. The $>$ entries are the counting estimate of Equation (2).

estimate and the enumeration can be halved by duality: orthogonal complement matches each orbit of d-dimensional subspaces with one of dimension $lm - d$ in equal number, so only the dimensions $d \leq \lfloor lm/2 \rfloor$ need be enumerated.

More invariants

Clearing *visited* per dimension exploits a single G-invariant, the subspace dimension: two subspaces in the same orbit always have the same dimension, so the set never needs to hold subspaces of different dimensions simultaneously. The same idea applies to *any* G-invariant. Fixing a value (or a tuple of values) of one or more invariants, keeping in *visited* only the representatives realizing that value, and sweeping the values one at a time replaces a single large hash set with several small ones and never merges orbits across values, since equivalent subspaces agree on every invariant. We do not currently exploit invariants beyond the dimension, as enumeration memory is not the bottleneck of our results; the invariants of G available for each family include the following.

Matrix multiplication ($A = \mathbb{F}_q^{l\times m}$, $G = (\mathrm{GL}_l(\mathbb{F}_q) \times \mathrm{GL}_m(\mathbb{F}_q)) \rtimes C_2$).

- **Rank distribution:** $\rho_i(S) = |\{M \in S : \operatorname{rank} M = i\}|$ for $i = 0, \ldots, \min(l, m)$. The sandwich action $M \mapsto PMQ^{-1}$ and the transpose preserve the rank of each M, so ρ is constant on orbits.
- **Point profiles:** let $B_0, \ldots, B_{d-1}$ be a basis of S. The left profile is $\lambda_i(S) = |\{x \in \mathbb{F}_q^l : \operatorname{rank}[x^\top B_0, \ldots, x^\top B_{d-1}] = i\}|$ for $i = 0, \ldots, m$, and the right profile is $\mu_i(S) = |\{y \in \mathbb{F}_q^m : \operatorname{rank}[B_0 y, \ldots, B_{d-1} y] = i\}|$ for $i = 0, \ldots, l$. Left multiplication preserves λ, right multiplication preserves μ, and a change of basis of S preserves both; when $l = m = n$ the transpose swaps them, so the unordered pair $\{\lambda, \mu\}$ is invariant.

Full polynomial multiplication ($A =$ degree-$(N-1)$ binary forms, $G_{\text{full}} = \mathrm{P\Gamma L}_2(\mathbb{F}_q)$). A nonzero form is an effective degree-$(N-1)$ divisor on $\mathbb{P}^1$, and $\mathrm{P\Gamma L}_2$ permutes the closed points of $\mathbb{P}^1$ preserving their degree.

- **Factorization-type distribution** (the analog of the rank distribution): for a nonzero $f \in S$, let $\tau(f)$ be the multiset of pairs $(\deg P, \operatorname{ord}_P f)$ over the closed points P in the support of $\operatorname{div}(f)$ — equivalently, the multiset of (degree, multiplicity) of the $\mathbb{F}_q$-irreducible factors of f. Every $g \in \mathrm{P\Gamma L}_2$ sends f to a form with the same type, so $\rho_\tau(S) = |\{f \in S : \tau(f) = \tau\}|$ is invariant.
- **Point profile** (the analog of the point profiles): for a closed point P and $k \geq 1$, let $S_{P,\geq k} = \{f \in S : \operatorname{ord}_P f \geq k\}$; the local vanishing profile is the codimension sequence $v_P(S) = (\dim S - \dim S_{P,\geq k})_{k\geq 1}$. Since $\mathrm{P\Gamma L}_2$ permutes the degree-δ closed points, for each δ the multiset $\{v_P(S) : \deg P = \delta\}$ is invariant; its $k = 1$ term is the rank of the evaluation map from S to the fiber at P. The projective action fuses 0, ∞, and every other point into one $\mathbb{P}^1$, so here there is a single point profile rather than the left/right pair of matrix multiplication.

Cyclic polynomial multiplication ($A = R = \mathbb{F}_q[x]/(x^N - 1)$, $G_{\text{cyc}} = (R^* \rtimes \mathrm{Aut}(R) \rtimes \mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p))/\mathbb{F}_q^*$). By the Chinese remainder theorem $R \cong \prod_{i=1}^{t} \mathbb{F}_{q^{d_i}}$ over the q-cyclotomic cosets of $\mathbb{Z}/N$, with projections $\pi_i \colon R \to \mathbb{F}_{q^{d_i}}$ (evaluation at the N-th roots of unity). G_{cyc} scales each π_i by a unit, permutes the equal-degree factors, and twists them by field automorphisms, so it preserves the degree of each factor and the zero pattern of $a \in R$.

- **Support-type distribution** (the analog of the rank distribution): for $a \in S$, let $\mathrm{supp}(a) = \{d_i : \pi_i(a) \neq 0\}$ be the multiset of degrees of the factors on which a is supported. Then $\rho_\sigma(S) = |\{a \in S : \mathrm{supp}(a) = \sigma\}|$ is invariant.
- **Projection-dimension profile** (the cyclic point profile): for each factor $\dim_{\mathbb{F}_q} \pi_i(S) \in \{0, \ldots, d_i\}$ (a unit scales $\pi_i(S)$ within $\mathbb{F}_{q^{d_i}}$, preserving its $\mathbb{F}_q$-dimension); for each δ the multiset $\{\dim_{\mathbb{F}_q} \pi_i(S) : d_i = \delta\}$ is invariant. These factors are exactly the evaluation points at the N-th roots of unity.

All three point profiles are the same construction — evaluate the subspace at the points the symmetry group permutes and record the resulting rank distribution — and differ only in the point set: the left/right vector spaces for matrix multiplication, the projective line $\mathbb{P}^1$ for full polynomial multiplication, and the N-th roots of unity for cyclic convolution.

D Toy example in detail: $\mathbf{R}_{\mathbb{F}_2}(\langle 2,2,2\rangle) \geq 7$

We illustrate the dynamic program on $\langle 2,2,2\rangle$ over $\mathbb{F}_2$ in detail, where A is the space of 2×2 matrices and the symmetry group is $G = (\mathrm{GL}_2(\mathbb{F}_2) \times \mathrm{GL}_2(\mathbb{F}_2)) \rtimes C_2$ acting by $X \mapsto PXQ^{-1}$ and by transposition. There are ten orbits of constraint subspaces. The program processes them from the most constrained to the unconstrained tensor, assigning each a lower bound by whichever of the four techniques of Section 6 succeeds first. We write a_{ij} for the free entries of A that survive the constraints, and the constrained tensor is $\sum_{i,j,k} a_{ij} \otimes b_{jk} \otimes c_{ki}$ restricted to them.

Orbit 0: $\{a_{00} = 0,\ a_{01} = 0,\ a_{10} = 0,\ a_{11} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$. The constrained tensor is $T_0 = 0$, so $\mathbf{R}(T_0) = 0$.

Orbit 1: $\{a_{00} = 0,\ a_{01} = 0,\ a_{10} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ 0 & a_{11} \end{pmatrix}$, with tensor $T_1 = a_{11} \otimes b_{10} \otimes c_{01} + a_{11} \otimes b_{11} \otimes c_{11}$. Flattening the A and B factors into one yields a matrix of rank 2, so $\mathbf{R}(T_1) \geq 2$.

Orbit 2: $\{a_{00} = 0,\ a_{01} + a_{10} = 0,\ a_{11} = 0\}$

$A = \begin{pmatrix} 0 & a_{01} \\ a_{01} & 0 \end{pmatrix}$, with tensor $T_2 = a_{01} \otimes b_{00} \otimes c_{01} + a_{01} \otimes b_{01} \otimes c_{11} + a_{01} \otimes b_{10} \otimes c_{00} + a_{01} \otimes b_{11} \otimes c_{10}$. Flattening yields a matrix of rank 4, so $\mathbf{R}(T_2) \geq 4$.

Orbit 3: $\{a_{00} = 0,\ a_{01} = 0\}$

$A = \begin{pmatrix} 0 & 0 \\ a_{10} & a_{11} \end{pmatrix}$, with tensor $T_3 = a_{10} \otimes b_{00} \otimes c_{01} + a_{10} \otimes b_{01} \otimes c_{11} + a_{11} \otimes b_{10} \otimes c_{01} + a_{11} \otimes b_{11} \otimes c_{11}$. Flattening yields rank 4, so $\mathbf{R}(T_3) \geq 4$.

Orbit 4: $\{a_{00} = 0,\ a_{01} + a_{10} = 0\}$

$A = \begin{pmatrix} 0 & a_{01} \\ a_{01} & a_{11} \end{pmatrix}$, with tensor

$T_4 = a_{01} \otimes b_{00} \otimes c_{01} + a_{01} \otimes b_{01} \otimes c_{11} + a_{01} \otimes b_{10} \otimes c_{00} + a_{01} \otimes b_{11} \otimes c_{10} + a_{11} \otimes b_{10} \otimes c_{01} + a_{11} \otimes b_{11} \otimes c_{11}$.

Flattening yields only 4, so we invoke the forced-product technique (Section 6), which automates the substitution of Hopcroft and Kerr stated in Lemma 2.

Fix an optimal decomposition $T_4 = \sum_\lambda u_\lambda \otimes v_\lambda \otimes w_\lambda$ with $r = \mathbf{R}(T_4)$ terms. Grouping T_4 along the C factor exposes its slices as forms in $A \otimes B$: the slice at c_{00} is the single product $a_{01} \otimes b_{10}$ and the slice at c_{10} is the single product $a_{01} \otimes b_{11}$, and these two are linearly independent (so $s = 2$). By Lemma 2 we may assume the decomposition computes them literally, say $u_0 \otimes v_0 = a_{01} \otimes b_{10}$ and $u_1 \otimes v_1 = a_{01} \otimes b_{11}$.

Now strip these two terms from T_4. The lemma pins down their $A \otimes B$ parts but not their C-components w_0, w_1: each must match the slice it computes along c_{00} (resp. c_{10}), but its coordinates along the remaining basis vectors c_{01}, c_{11} are unconstrained. Over $\mathbb{F}_2$ this leaves four unknown bits $\mu_0, \ldots, \mu_3$, and the residual is

$$T_4' = T_4 + a_{01} \otimes b_{10} \otimes (c_{00} + \mu_0 c_{01} + \mu_1 c_{11}) + a_{01} \otimes b_{11} \otimes (c_{10} + \mu_2 c_{01} + \mu_3 c_{11}), \qquad \mu_i \in \{0, 1\}.$$

Enumerating all $2^4 = 16$ assignments of μ and flattening each gives $\mathbf{R}(T_4') \geq 4$ in every case, so $\mathbf{R}(T_4) \geq 4 + 2 = 6$.

Orbit 5: $\{a_{00} = 0,\ a_{11} = 0\}$

$A = \begin{pmatrix} 0 & a_{01} \\ a_{10} & 0 \end{pmatrix}$, with tensor $T_5 = a_{01} \otimes b_{10} \otimes c_{00} + a_{01} \otimes b_{11} \otimes c_{10} + a_{10} \otimes b_{00} \otimes c_{01} + a_{10} \otimes b_{01} \otimes c_{11}$. Flattening yields rank 4, so $\mathbf{R}(T_5) \geq 4$.

Orbit 6: $\{a_{01} + a_{10} = 0,\ a_{00} + a_{01} + a_{11} = 0\}$

$A = \begin{pmatrix} a_{00} & a_{01} \\ a_{01} & a_{00} + a_{01} \end{pmatrix}$. Flattening gives only 4. Here, we try to get a better lower bound using substitution with backtracking (Section 6). Fix an optimal decomposition $T_6 = \sum_\lambda u_\lambda \otimes v_\lambda \otimes w_\lambda$; as in the hypothesis of Lemma 3, we may take each u_λ to lie in the constrained space spanned by the free entries $\{a_{00}, a_{01}\}$, since projecting the u_λ onto that space changes no term's contribution on the constrained tensor and can only shorten the decomposition. The only nonzero linear forms in a_{00}, a_{01} over $\mathbb{F}_2$ are a_{00}, a_{01}, and $a_{00} + a_{01}$, so every nonzero u_λ is one of these three. The flattening bound just computed shows the decomposition has at least 4 terms, so by pigeonhole one of the three forms occurs as some u_λ at least twice, and setting it to zero kills at least two terms:

- $a_{00} = 0$ leaves $\begin{pmatrix} 0 & a_{01} \\ a_{01} & a_{01} \end{pmatrix}$, Orbit 2 after adding row 0 to row 1;
- $a_{01} = 0$ leaves $\begin{pmatrix} a_{00} & 0 \\ 0 & a_{00} \end{pmatrix}$, Orbit 2 after swapping the rows;

- $a_{00} + a_{01} = 0$ leaves $\begin{pmatrix} a_{00} & a_{00} \\ a_{00} & 0 \end{pmatrix}$, Orbit 2 after adding row 1 to row 0.

This pigeonhole argument of the lower bound 6 can be automated by the backtracking below.

- a_{00}: 6 not met; go deeper.
  - a_{00}, a_{00}: set $a_{00} = 0$, reaching Orbit 2; $\mathbf{R}(T_6) \geq \mathbf{R}(T_2) + 2 \geq 6$. Proved; backtrack.
  - a_{00}, a_{01}: 6 not met; go deeper.
    - a_{00}, a_{01}, a_{01}: set $a_{01} = 0$, reaching Orbit 2; $\mathbf{R}(T_6) \geq 6$. Proved; backtrack.
    - $a_{00}, a_{01}, a_{00} + a_{01}$: 6 not met; go deeper.
      - $a_{00}, a_{01}, a_{00} + a_{01}, a_{00} + a_{01}$: set $a_{00} + a_{01} = 0$, reaching Orbit 2; $\mathbf{R}(T_6) \geq 6$. Proved; backtrack.
  - $a_{00}, a_{00} + a_{01}$: 6 not met; go deeper.
    - $a_{00}, a_{00} + a_{01}, a_{00} + a_{01}$: set $a_{00} + a_{01} = 0$, reaching Orbit 2; $\mathbf{R}(T_6) \geq 6$. Proved; backtrack.
- a_{01}: 6 not met; go deeper.
  - a_{01}, a_{01}: set $a_{01} = 0$, reaching Orbit 2; $\mathbf{R}(T_6) \geq 6$. Proved; backtrack.
  - $a_{01}, a_{00} + a_{01}$: 6 not met; go deeper.
    - $a_{01}, a_{00} + a_{01}, a_{00} + a_{01}$: set $a_{00} + a_{01} = 0$, reaching Orbit 2; $\mathbf{R}(T_6) \geq 6$. Proved; backtrack.
- $a_{00} + a_{01}$: 6 not met; go deeper.
  - $a_{00} + a_{01}, a_{00} + a_{01}$: set $a_{00} + a_{01} = 0$, reaching Orbit 2; $\mathbf{R}(T_6) \geq 6$. Proved; backtrack.

Every branch reaches the target, so $\mathbf{R}(T_6) \geq 6$.

Orbit 7: $\{a_{00} = 0\}$

$A = \begin{pmatrix} 0 & a_{01} \\ a_{10} & a_{11} \end{pmatrix}$. Adding the constraint $a_{01} + a_{10} = 0$ lands in Orbit 4, so degenerate reduction (Section 6) gives $\mathbf{R}(T_7) \geq \mathbf{R}(T_4) \geq 6$.

Orbit 8: $\{a_{01} + a_{10} = 0\}$

$A = \begin{pmatrix} a_{00} & a_{01} \\ a_{01} & a_{11} \end{pmatrix}$. Adding the constraint $a_{00} = 0$ lands in Orbit 4, so $\mathbf{R}(T_8) \geq \mathbf{R}(T_4) \geq 6$.

Orbit 9: $\{\,\}$

$A = \begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix}$ is the unconstrained tensor $T_9 = \sum_{i,j,k} a_{ij} \otimes b_{jk} \otimes c_{ki} = \langle 2, 2, 2 \rangle$. We apply substitution with backtracking. Fix an optimal decomposition $T_9 = \sum_\lambda u_\lambda \otimes v_\lambda \otimes w_\lambda$. If a_{00}

appears as some u_λ, setting $a_{00} = 0$ lands in Orbit 7 and gives $\mathbf{R}(T_9) \geq \mathbf{R}(T_7) + 1 \geq 7$; by the sandwich symmetry the same holds when the form is any of a_{01}, a_{10}, a_{11}, $a_{00} + a_{01}$, $a_{10} + a_{11}$, $a_{00} + a_{10}$, $a_{01} + a_{11}$, or $a_{00} + a_{01} + a_{10} + a_{11}$. Otherwise the form is one of $a_{01} + a_{10}$, $a_{00} + a_{11}$, $a_{00} + a_{01} + a_{10}$, $a_{00} + a_{01} + a_{11}$, $a_{00} + a_{10} + a_{11}$, or $a_{01} + a_{10} + a_{11}$, and setting it to zero lands in Orbit 8, giving $\mathbf{R}(T_9) \geq \mathbf{R}(T_8) + 1 \geq 7$. These exhaust the nonzero forms, so $\mathbf{R}_{\mathbb{F}_2}(\langle 2, 2, 2 \rangle) = \mathbf{R}(T_9) \geq 7$.